

# Dart Programming

# USING BASIC WIDGETS

- **Scaffold:** the Scaffold widget implements the basic Material Design visual layout, allowing you to easily add various widgets such as AppBar, and more.
- **AppBar:** The AppBar widget usually contains the standard title, toolbar, leading, and actions properties (along with buttons), as well as many customization options.
  - **title** The title property is typically implemented with a Text widget. You can customize it with other widgets such as a DropdownButton widget.
  - **leading** The leading property is displayed before the title property. Usually this is an IconButton or BackButton.
  - **actions** The actions property is displayed to the right of the title property. It's a list of widgets aligned to the upper right of an AppBar widget usually with an IconButton or PopupMenuButton.
  - **flexibleSpace** The flexibleSpace property is stacked behind the Toolbar or TabBar widget. A background image is commonly applied to the flexibleSpace property, but any widget, such as an Icon, could be used.

# USING BASIC WIDGETS

- **SafeArea** The SafeArea widget is necessary for today's devices such as the iPhone X or Android devices with a notch (a partial cut-out obscuring the screen usually located on the top portion of the device). The SafeArea widget automatically adds sufficient padding to the child widget to avoid intrusions by the operating system. You can optionally pass a minimum amount of padding or a Boolean value to not enforce padding on the top, bottom, left, or right.
- **Container** The Container widget is a commonly used widget that allows customization of its child widget. You can easily add properties such as color, width, height, padding, margin, border, constraint, alignment, transform, and many others. The child property is optional, and the Container widget can be used as an empty placeholder (invisible) to add space between widgets.
- **Text** The Text widget is used to display a string of characters. The Text constructor takes the arguments string, style, maxLines, overflow, textAlign, and others. A constructor is how the arguments are passed to initialize and customize the Text widget.

# USING BASIC WIDGETS

- **RichText** The RichText widget is a great way to display text using multiple styles. The RichText widget takes TextSpans as children to style different parts of the strings.
- **Column** A Column widget displays its children vertically. It takes a children property containing an array of List<Widget>, meaning you can add multiple widgets. The children align vertically without taking up the full height of the screen. Each child widget can be embedded in an Expanded widget to fill the available space. CrossAxisAlignment, MainAxisAlignment, and MainAxisSize can be used to align and size how much space is occupied on the main axis.
- **Row** A Row widget displays its children horizontally. It takes a children property containing an array of List<Widget>. The same properties that the Column contains are applied to the Row widget with the exception that the alignment is horizontal, not vertical.
- **Buttons** There are a variety of buttons to choose from for different situations such as RaisedButton, FloatingActionButton, FlatButton, IconButton, PopupMenuButton, and ButtonBar.

# Adding AppBar Widgets

- Open the main.dart file. Change the primarySwatch property from blue to lightGreen. primarySwatch: Colors.lightGreen,
- Open the home.dart file. Start by customizing the AppBar widget properties.
- Add to the AppBar a leading IconButton. If you override the leading property, it is usually an IconButton or BackButton.
  - leading: IconButton(
    - icon: Icon(Icons.menu),
    - onPressed: () { },
    - ),
- The title property is usually a Text widget, but it can be customized with other widgets such as a DropdownButton., add the Text widget with a value of 'Home'.
  - title: Text('Home'),

# Adding AppBar Widgets

- The actions property takes a list of widgets; add two IconButton widgets.
  - actions: <Widget>[
  - IconButton(
  - icon: Icon(Icons.search),
  - onPressed: () {},
  - ),
  - IconButton(
  - icon: Icon(Icons.more\_vert),
  - onPressed: () {},
  - ),
  - ],



# Adding AppBar Widgets

- Because you are using an Icon for the flexibleSpace property, let's add a SafeArea and an Icon as a child.
  - flexibleSpace: SafeArea(
    - child: Icon(
      - Icons.photo\_camera,
      - size: 75.0,
      - color: Colors.white70,
      - ),
      - ),



# Adding AppBar Widgets

- Add a PreferredSize for the bottom property with a Container for a child.
  - bottom: PreferredSize(
    - child: Container(
      - color: Colors.lightGreen.shade100,
      - height: 75.0,
      - width: double.infinity,
      - child: Center(
        - child: Text('Bottom'),
        - ),
        - ),
      - preferredSize: Size.fromHeight(75.0),
      - ),



# Adding a SafeArea to the Body

- Add a Padding widget to the body property with a SafeArea as a child.
- The SingleChildScrollView allows the user to scroll and view hidden widgets; otherwise, the user sees a yellow and black bar conveying that the widgets are overflowing.
  - body: Padding(
    - padding: EdgeInsets.all(16.0),
    - child: SafeArea(
      - child: SingleChildScrollView(
        - child: Column(
          - children: <Widget>[
          - ],
          - ),
          - ),
          - ),
          - ),

# Container

- The Container widget has an optional child widget property and can be used as a decorated widget with a custom border, color, constraint, alignment, transform (such as rotating the widget), and more.
- This widget can be utilized as an empty placeholder (invisible), and if a child is omitted, it sizes to the full available screen size.



# Adding a Container

- Add to the body property a Padding widget with the child property set to a SafeArea widget.
- Add to the SafeArea child a SingleChildScrollView. Add to the SingleChildScrollView child a Column.
- For the Column children, add the call to the
- ContainerWithBoxDecorationWidget() widget class, which you will create next. Make sure the widget class uses the const keyword to take advantage of caching (performance).
  - body: Padding(
    - padding: EdgeInsets.all(16.0),
    - child: SafeArea(
      - child: SingleChildScrollView(
        - child: Column(
          - children: <Widget>[
          - const ContainerWithBoxDecorationWidget(),],
          - ),),),),

# Adding a Container

- Add to the body property a Padding widget with the child property set to a SafeArea widget.
- Add to the SafeArea child a SingleChildScrollView. Add to the SingleChildScrollView child a Column.
- For the Column children, add the call to the
- ContainerWithBoxDecorationWidget() widget class, which you will create next. Make sure the widget class uses the const keyword to take advantage of caching (performance).
  - body: Padding(
    - padding: EdgeInsets.all(16.0),
    - child: SafeArea(
      - child: SingleChildScrollView(
        - child: Column(
          - children: <Widget>[
          - const ContainerWithBoxDecorationWidget(),],
          - ),),),),

# Adding a Container

- Create the `ContainerWithBoxDecorationWidget()` widget class after class `Home` extends `StatelessWidget` {...}.
- The widget class will return a `Widget`. Note that when you refactor by
- creating widget classes, they are of type `StatelessWidget` unless you specify to use a `StatefulWidget`.
  - `class ContainerWithBoxDecorationWidget extends StatelessWidget {`
  - `const ContainerWithBoxDecorationWidget({`
  - `Key key,`
  - `}) : super(key: key);`
  - `@override`
  - `Widget build(BuildContext context) {`
  - `return Column(`
  - `children: <Widget>[`
  - `Container(),`
  - `],`
  - `);}}`

# Adding a Container

- Start adding properties to the Container by adding a height of 175.0 pixels. Note the comma after the number, which separates properties and helps to keep the Dart code formatted.
- Go to the next line to add the decoration property, which accepts a BoxDecoration class.
- The BoxDecoration class provides different ways to draw a box, and in this case, you are adding a BorderRadius class to the bottomLeft and bottomRight of the Container.
  - Container(
    - height: 100.0,
    - decoration: BoxDecoration(),
    - ),



# Adding a Container

- Using the named constructor BorderRadius.only() allows you to control the sides to draw round corners. BoxDecoration(
  - borderRadius: BorderRadius.only(
    - bottomLeft: Radius.circular(100.0),
    - bottomRight: Radius.circular(10.0),
    - ),
    - ),

# Adding a Container

- The BoxDecoration also supports a gradient property. We are using a LinearGradient, but We could also have used a RadialGradient.
- The LinearGradient displays the gradient colors linearly, and the RadialGradient displays the gradient colors in a circular manner.
- The begin and end properties allows to choose the start and end positions for the gradient by using the AlignmentGeometry class.
- AlignmentGeometry is a base class for Alignment that allows direction-aware resolution. We have many directions to choose from such as Alignment.bottomLeft, Alignment.centerRight, and more.
- The colors property requires a List of Color types, List<Color>. The list of Colors is entered within square brackets separated by commas.
  - gradient: LinearGradient(
    - begin: Alignment.topCenter,
    - end: Alignment.bottomCenter,
    - colors: [
      - Colors.white,
      - Colors.lightGreen.shade500,],),

# Adding a Container

- The `boxShadow` property is a great way to customize a shadow, and it takes a list of `BoxShadows`, called `List<BoxShadow>`.
- For the `BoxShadow`, set the `color`, `blurRadius`, and `offset` properties.
  - `boxShadow: [`
  - `BoxShadow(`
  - `color: Colors.white,`
  - `blurRadius: 10.0,`
  - `offset: Offset(0.0, 10.0),`
  - `)`
  - `],`

# Adding a Container

- The last part of the Container is to add a child Text widget wrapped by a Center widget.
- The Center widget allows you to center the child widget on the screen.
- Add a Center widget as a child of the Container, and add to the Center widget child a Text widget with the string Container.
  - child: Center(
    - child: Text('Container'),
    - ),

# Text

- The Text constructor takes the arguments string, style, maxLines, overflow, textAlign, and others.
  - Text(
    - 'Flutter World for Mobile',
    - style: TextStyle(
      - fontSize: 24.0,
      - color: Colors.deepPurple,
      - decoration: TextDecoration.underline,
      - decorationColor: Colors.deepPurpleAccent,
      - decorationStyle: TextDecorationStyle.dotted,
      - fontStyle: FontStyle.italic,
      - fontWeight: FontWeight.bold,)),
      - maxLines: 4,
      - overflow: TextOverflow.ellipsis,
      - textAlign: TextAlign.justify,)),

# RichText

- The RichText widget is a great way to display text using multiple styles.
- The RichText widget takes TextSpan as children to style different parts of the strings
- By using the RichText widget and combining different TextSpan objects, you create rich-text formatting like with a word processor.
  - child: Center(
    - child: RichText(
      - text: TextSpan(
        - text: 'Flutter World',
        - style: TextStyle(
          - fontSize: 24.0,
          - color: Colors.deepPurple,
          - decoration: TextDecoration.underline,
          - decorationColor: Colors.deepPurpleAccent,
          - decorationStyle: TextDecorationStyle.dotted,
          - fontStyle: FontStyle.italic,
          - fontWeight: FontWeight.normal,
    - ),



# RichText

- children: <TextSpan>[
- TextSpan(
- text: ' for',
- ),
- TextSpan(
- text: ' Mobile',
- style: TextStyle(
- color: Colors.deepOrange,
- fontStyle: FontStyle.normal,
- fontWeight: FontWeight.bold),
- ),
- ],
- ),
- ),
- ),

# Column

- A Column widget displays its children vertically.
- It takes a children property containing an array of List<Widget>.
- The children align vertically without taking up the full height of the screen.
- Each child widget can be embedded in an Expanded widget to fill available space.
- We can use CrossAxisAlignment, MainAxisAlignment, and MainAxisSize to align and size how much space is occupied on the main axis.
  - Column(
    - crossAxisAlignment: CrossAxisAlignment.center,
    - mainAxisAlignment: MainAxisAlignment.spaceEvenly,
    - mainAxisSize: MainAxisSize.max,
    - children: <Widget>[
      - Text('Column 1'),
      - Divider(),
      - Text('Column 2'),
      - Divider(),
      - Text('Column 3'),],),

# Row

- A Row widget displays its children horizontally.
- It takes a children property containing an array of List<Widget>.
- The same properties that the Column contains are applied to the Row widget with the exception that the alignment is horizontal, not vertical.
  - Row(
    - crossAxisAlignment: CrossAxisAlignment.start,
    - mainAxisAlignment: MainAxisAlignment.
    - spaceEvenly,
    - mainAxisAlignment: MainAxisAlignment.max,
    - children: <Widget>[
      - Row(
        - children: <Widget>[
          - Text('Row 1'),
          - Padding(padding: EdgeInsets.all(16.0)),
          - Text('Row 2'),
          - Padding(padding: EdgeInsets.all(16.0)),
          - Text('Row 3'),
          - ],),],),

# Column and Row Nesting

- A great way to create unique layouts is to combine Column and Row widgets for individual needs.
- Imagine having a journal page with Text in a Column with a nested Row containing a list of images.
- Add a Row widget inside the Column widget.
- Use `mainAxisAlignment: MainAxisAlignment.spaceEvenly` and add three Text widgets.



# Column and Row Nesting

- Column(
  - crossAxisAlignment: CrossAxisAlignment.start,
  - mainAxisAlignment: MainAxisAlignment.
  - spaceEvenly,
  - mainAxisSize: MainAxisSize.max,
  - children: <Widget>[
    - Text('Columns and Row Nesting 1'),
    - Text('Columns and Row Nesting 2'),
    - Text('Columns and Row Nesting 3'),
    - Padding(padding: EdgeInsets.all(16.0)),
    - Row(
      - mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      - children: <Widget>[
        - Text('Row Nesting 1'),
        - Text('Row Nesting 2'),
        - Text('Row Nesting 3'),
        - ],
        - ),],

# Buttons

- **FloatingActionButton**

- The FloatingActionButton widget is usually placed on the bottom right or center of the main screen in the Scaffold floatingActionButton property.
- Use the FloatingActionButtonLocation widget to either dock (notch) or float above the navigation bar.
- To dock a button to the navigation bar, use the BottomAppBar widget.
  - floatingActionButtonLocation: FloatingActionButtonLocation.endDocked,
  - floatingActionButton: FloatingActionButton(  - onPressed: () {},
  - child: Icon(Icons.play\_arrow),
  - backgroundColor: Colors.lightGreen.shade100,),
  - bottomNavigationBar: BottomAppBar(  - hasNotch: true,
  - color: Colors.lightGreen.shade100,
  - child: Row(  - mainAxisAlignment: MainAxisAlignment.spaceEvenly,
  - children: <Widget>[
  - Icon(Icons.pause),
  - Icon(Icons.stop),
  - Icon(Icons.access\_time),
  - Padding(padding: EdgeInsets.all(32.0)),],),)



# Buttons

- **FlatButton**

- The FlatButton widget is the most minimalist button used; it displays a text label without any borders or elevation (shadow).
- Since the text label is a widget, you could use an Icon widget instead or another widget to customize the button. color, highlightColor, splashColor, textColor, and other properties can be customized.
- // Default - left button
- FlatButton(
  - onPressed: () {},
  - child: Text('Flag'),
  - ),
- // Customize - right button
- FlatButton(
  - onPressed: () {},
  - child: Icon(Icons.flag),
  - color: Colors.lightGreen,
  - textColor: Colors.white,)

# Buttons

- **RaisedButton**

- The RaisedButton widget adds a dimension, and the elevation (shadow) increases when the user presses the button.
  - // Default - left button
  - RaisedButton(
    - onPressed: () {},
    - child: Text('Save'),
    - ),
  - // Customize - right button
  - RaisedButton(
    - onPressed: () {},
    - child: Icon(Icons.save),
    - color: Colors.lightGreen,
    - ),

# Buttons

- **IconButton**

- The IconButton widget uses an Icon widget on a Material Component widget that reacts to touches by filling with color (ink).
- The combination creates a nice tap effect, giving the user feedback that an action has started.
  - // Default - left button
  - IconButton(
    - onPressed: () {},
    - icon: Icon(Icons.flight),
    - ),
  - // Customize - right button
  - IconButton(
    - onPressed: () {},
    - icon: Icon(Icons.flight),
    - iconSize: 42.0,
    - color: Colors.white,
    - tooltip: 'Flight',
    - ),

# Buttons

- **AppBar**

- The AppBar widget aligns buttons horizontally. In this example, the AppBar widget is a child of a Container widget to give it a background color.
- Container(
  - color: Colors.white70,
  - child: AppBar(
    - alignment: MainAxisAlignment.spaceEvenly,
    - children: <Widget>[
      - IconButton(
        - icon: Icon(Icons.map),
        - onPressed: () {},),
      - IconButton(
        - icon: Icon(Icons.airport\_shuttle),
        - onPressed: () {},),
      - IconButton(
        - icon: Icon(Icons.brush),
        - onPressed: () {},),),),

# USING IMAGES AND ICONS

- Images can make an app look tremendous or ugly depending on the quality of the artwork. Images, icons, and other resources are commonly embedded in an app.
  - **AssetBundle**
    - The AssetBundle class provides access to custom resources such as images, fonts, audio, data files, and more.
    - Before a Flutter app can use a resource, you must declare it in the pubspec.yaml file.
    - // pubspec.yaml file to edit
    - # To add assets to your application, add an assets section, like this:
    - Make sure you end the directory name with a forward slash, /.
      - assets:
      - — assets/images/
      - — assets/data/

# USING IMAGES AND ICONS

- **Image**

- The Image widget displays an image from a local or URL (web) source. To load an Image widget,
- there are a few different constructors to use.
  - Image()—Retrieves image from an ImageProvider class
  - Image.asset()—Retrieves image from an AssetBundle class using a key
  - Image.file()—Retrieves image from a File class
  - Image.memory()—Retrieves image from a Uint8List class
  - Image.network()—Retrieves image from a URL path



# USING IMAGES AND ICONS

- // Image - on the left side
- Image(
  - image: AssetImage("assets/images/logo.png"),
  - fit: BoxFit.cover,
  - ),
- // Image from a URL - on the right side
- Image.network(
  - 'https://flutter.io/images/catalog-widget-placeholder.png',
  - ),



- If you add color to the image, it colonizes the image portion and leaves any transparencies alone, giving a silhouette look

# USING IMAGES AND ICONS

- **Icon**

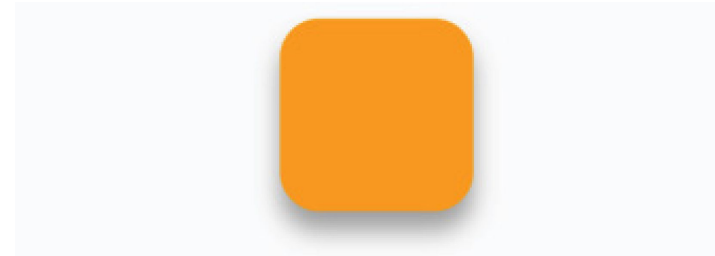
- The Icon widget is drawn with a glyph from a font described in IconData.
- Flutter's icons.dart file has the full list of icons available from the font MaterialIcons.
- A great way to add custom icons is to add to the AssetBundle fonts containing glyphs.
- The Icon widget allows you to change the Icon widget's color, size, and other properties
  - Icon(
    - Icons.brush,
    - color: Colors.lightBlue,
    - size: 48.0,
    - ),

# USING DECORATORS

- Decorators help to convey a message depending on the user's action or customize the look and feel of a widget.
- There are different types of decorators for each task.
  - **Decoration**—The base class to define other decorations.
  - **BoxDecoration**—Provides many ways to draw a box with border, body, and boxShadow.
  - **InputDecoration**—Used in TextField and TextFormField to customize the border, label, icon, and styles. This is a great way to give the user feedback on data entry, specifying a hint, an error, an alert icon, and more.

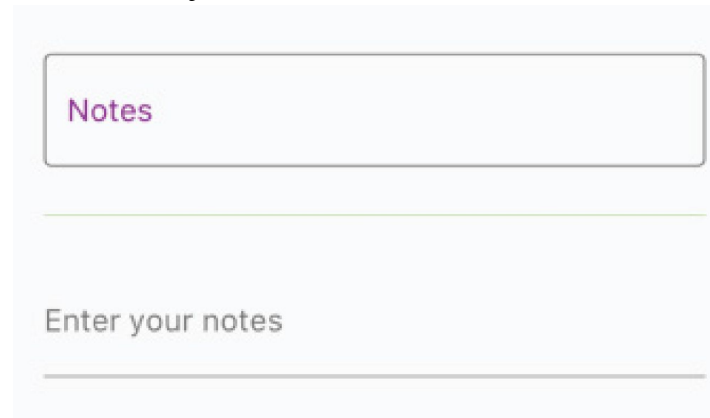
# USING DECORATORS

- A **BoxDecoration** class is a great way to customize a Container widget to create
- shapes by setting the borderRadius, color, gradient, and boxShadow properties.
  - Container(
    - height: 100.0,
    - width: 100.0,
    - decoration: BoxDecoration(
      - borderRadius: BorderRadius.all(Radius.circular(20.0)),
      - color: Colors.orange,
      - boxShadow: [
        - BoxShadow(
          - color: Colors.grey,
          - blurRadius: 10.0,
          - offset: Offset(0.0, 10.0),



# USING DECORATORS

- The **InputDecoration** class is used with a `TextField` or `TextFormField` to specify labels, borders, icons, hints, errors, and styles.
- This is helpful in communicating with the user as they enter data.
  - `TextField(`
  - `keyboardType: TextInputType.text,`
  - `style: TextStyle(`
  - `color: Colors.grey.shade800,`
  - `fontSize: 16.0),`
  - `decoration: InputDecoration(`
  - `labelText: "Notes",`
  - `labelStyle: TextStyle(color: Colors.purple),`
  - `border: OutlineInputBorder(),),),`
  - `// TextFormField`
  - `TextFormField(`
  - `decoration: InputDecoration(`
  - `labelText: 'Enter your notes',),),`



Notes

Enter your notes

# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- The Form widget is optional, but the benefits of using a Form widget are to validate each text field as a group.
- We can group TextFormField widgets to manually or automatically validate them.
- The TextFormField widget wraps a TextField widget to provide validation when enclosed in a Form widget.
- If all text fields pass the FormState validate method, then it returns true.
- If any text fields contain errors, it displays the appropriate error message for each text field, and the FormState validate method returns false.
- This process gives you the ability to use FormState to check for any validation errors instead of checking each text field for errors and not allowing the posting of invalid data.
- The Form widget needs a unique key to identify it and is created by using GlobalKey. This GlobalKey value is unique across the entire app.



# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- Create a new Flutter project. For this project, you need to create only the pages folder.
  - Open the home.dart file and add to the body a SafeArea widget with Column as a child.
  - In the Column children, add the Form() widget.
    - body: SafeArea(
      - child: Column(
        - children: <Widget>[
          - Form(),
  - },
  - ),
  - ),
- Create the Order class after class \_HomeState extends State<Home> {...}. The Order class will hold item as a String value and quantity as an int value.
  - class \_HomeState extends State<Home> {
  - }
  - class Order {
  - String item;
  - int quantity;}

# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- After the class `_HomeState` extends `State<Home>` declaration and before `@override`, add the variables `_formStateKey` for the `GlobalKey` value and `_order` to initiate the `Order` class.
- Create the unique key for the form by using `GlobalKey<FormState>` and mark it `final` since it will not change.
  - `class _HomeState extends State<Home> {`
  - `final GlobalKey<FormState> _formStateKey = GlobalKey<FormState>();`
  - `Order _order = Order();`
- Create the `_validateItemRequired(String value)` method that accepts a `String` value.
- Use the ternary operator to check whether the value is set to `isEmpty`, and if yes, then return 'Item Required'. Otherwise, return `null`.
  - `String _validateItemRequired(String value) {`
  - `return value.isEmpty ? 'Item Required' : null;`
  - `}`

# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- Create the `_validateItemCount(String value)` method that accepts a String value.
- Use the ternary operator to convert String to int. Then check whether int is greater than zero; if it's not, return 'At least one Item is Required'.
  - `String _validateItemCount(String value) {`
  - `int _valueAsInteger = value.isEmpty ? 0 : int.tryParse(value);`
  - `return _valueAsInteger == 0 ? 'At least one Item is Required' : null;`
  - `}`
- Create the `_submitOrder()` method called by the FlatButton widget to check whether all TextFormField fields pass validation and call Form save() to gather values from all TextFormFields to the Order class.
  - `void _submitOrder() {`
  - `if(_formStateKey.currentState.validate()) {`
  - `_formStateKey.currentState.save();`
  - `print('Order Item: ${order.item}');`
  - `print('Order Quantity: ${order.quantity}');`
  - `}`
  - `}`

# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- Add to the Form() widget a private key variable called `_formStateKey`, set `autovalidate` to `true`, add `Padding` for the child property, and add `Column` as a child of `Padding`.
- Setting `autovalidate` to `true` allows the Form() widget to check validation for all fields as the user enters information and to display an appropriate message.
- If `autovalidate` is set to `false`, no validation happens until the `_formStateKey.currentState.validate()` method is manually called.
  - Form(
    - key: `_formStateKey`,
    - autovalidate: `true`,
    - child: `Padding`(
      - padding: `EdgeInsets.all(16.0)`,
      - child: `Column`(
        - children: `<Widget>[`
          - ],
          - ),
          - ),
          - ),

# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- Add two TextFormField widgets to the Column children list of Widget.
- The first TextFormField is an item description, and the second TextFormField is a quantity of items to order.
- Add an InputDecoration class with hintText and labelText for each TextFormField.
  - hintText: 'Espresso',
  - labelText: 'Item',
- Add a call for the validator and onSave methods. The validator method is called to validate characters as they are entered, and the onSave method is called by the Form save() method to gather values from each TextFormField.
- For the validator, pass the value entered in the TextFormField widget by naming the variable value inside parentheses and use the fat arrow syntax (=>) to call the method
  - validator: (value) => \_validateItemRequired(value),

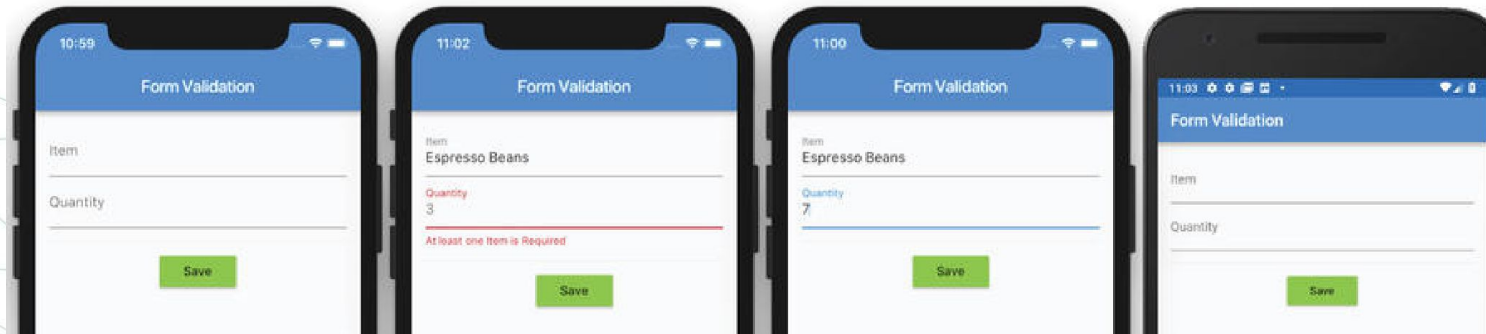
# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- The following code shows both TextFormField:
  - TextFormField(
    - decoration: InputDecoration(
      - hintText: 'Espresso',
      - labelText: 'Item',
    - ),
    - validator: (value) => \_validateItemRequired(value),
    - onSave: (value) => order.item = value,
    - ),
    - TextFormField(
      - decoration: InputDecoration(
        - hintText: '3',
        - labelText: 'Quantity',
      - ),
      - validator: (value) => \_validateItemCount(value),
      - onSave: (value) => order.quantity = int.tryParse(value),
      - ),



# USING THE FORM WIDGET TO VALIDATE TEXT FIELDS

- Add a Divider and a RaisedButton after the last TextFormField. For the onPressed, call the `_submitOrder()` method.
  - `Divider(height: 32.0,)`,
  - `RaisedButton(`
  - `child: Text('Save'),`
  - `color: Colors.lightGreen,`
  - `onPressed: () => _submitOrder(),`
  - `),`



# CHECKING ORIENTATION

- Under certain scenarios, knowing the device orientation helps in laying out the appropriate UI.
- There are two ways to figure out orientation, **MediaQuery.of(context).orientation** and **OrientationBuilder**.
  - A huge note on OrientationBuilder: it returns the amount of space available to the parent to figure out orientation.
  - This means it does not guarantee the actual device orientation.
  - We prefer using MediaQuery to obtain the actual device orientation because of its accuracy.

# CHECKING ORIENTATION

- Create a new Flutter project
- Open the home.dart file and add to the body a SafeArea with SingleChildScrollView as a child.
- Add Padding as a child of the SingleChildScrollView.
- Add a Column as a child of the Padding.
- In the Column children property, add the widget class called OrientationLayoutIconsWidget(), which we will create next.
  - body: SafeArea(
    - child: SingleChildScrollView(
      - child: Padding(
        - padding: EdgeInsets.all(16.0),
        - child: Column(
          - children: <Widget>[
          - const OrientationLayoutIconsWidget(),
          - ],
          - ),
          - ),
          - ),
          - ),

# CHECKING ORIENTATION

- Add the OrientationLayoutIconsWidget() widget class after class Home extends StatelessWidget {...}.
- The first variable to initialize is the current orientation by calling MediaQuery.of() after Widget build(BuildContext context).
  - class OrientationLayoutIconsWidget extends StatelessWidget {
  - const OrientationLayoutIconsWidget({
  - Key key,
  - }) : super(key: key);
  - @override
  - Widget build(BuildContext context) {
  - Orientation \_orientation = MediaQuery.of(context).orientation;
  - return Container();

# CHECKING ORIENTATION

- Add the OrientationLayoutIconsWidget() widget class after class Home extends StatelessWidget {...}.
- The first variable to initialize is the current orientation by calling MediaQuery.of() after Widget build(BuildContext context).
  - class OrientationLayoutIconsWidget extends StatelessWidget {
  - const OrientationLayoutIconsWidget({
  - Key key,
  - }) : super(key: key);
  - @override
  - Widget build(BuildContext context) {
  - Orientation \_orientation = MediaQuery.of(context).orientation;
  - return Container();

# CHECKING ORIENTATION

- Based on the current Orientation, you return a different layout of Icon widgets.
- Use a ternary operator to check whether Orientation is portrait, and if so, return a single Row icon.
- If Orientation is landscape, return a Row of two Icon widgets.
- Replace the current return Container() with the following code:
  - `return _orientation == Orientation.portrait ? Row( mainAxisAlignment: MainAxisAlignment.center,`
  - `children: <Widget>[`
  - `Icon(`
  - `Icons.school,`
  - `size: 48.0,)],)`
  - `: Row( mainAxisAlignment: MainAxisAlignment.center, children: <Widget>[`
  - `Icon(`
  - `Icons.school,`
  - `size: 48.0, ),`
  - `Icon(`
  - `Icons.brush,`
  - `size: 48.0,)],);`